

Lecture #5

Instructor: Yunseok Choi
(ys.choi@skku.edu)

Clustering

클러스터링 Clustering

- 데이터세트를 몇 개의 클러스터(그룹)로 나누는 것을 말함
 - 데이터 샘플들을 서로 비슷한 (가까운) 것끼리 그룹
- 학습 과정에서 보통 레이블을 사용하지 않기 때문에 '비지도학습'에 해당
- 클러스터링 수행 시 "가깝다" (혹은 비슷하다)를 정량적으로 정의해야 함 → **거리 함수** Distance Function
 - 유클리디언 거리 Euclidean Distance
 - 맨하탄 거리 (택시 거리) Manhattan Distance (Taxicab Distance)

거리함수 Distance Function

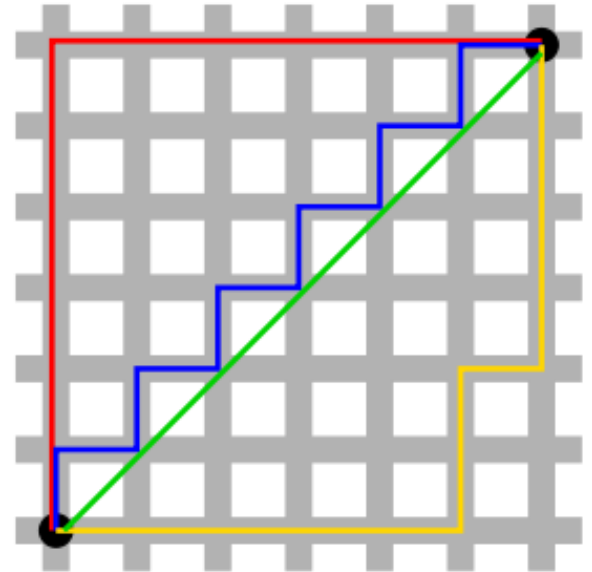
- 유클리디언 거리 Euclidean Distance

- 두 점의 최단 거리 (초록 선)

- 맨하탄 거리 (택시 거리)

Manhattan Distance (Taxicab Distance)

- 두 점이 존재하는 좌표에서
각 차원 별 거리의 합 (빨간 선)



소프트 vs. 하드 클러스터링

- 각 데이터가 **몇 개의 클러스터**에 속할 수 있는지?
 - 하드 클러스터링 :
각 데이터는 오직 한 개 클러스터에만 속할 수 있음
 - 소프트 클러스터링 :
각 데이터는 여러 개 클러스터에 속할 수 있음

클러스터링 TMI

- 어떤 데이터는 어떠한 클러스터에도 속하지 않을 수도 있음
(혼자 그룹)
→ 이상치(outlier) 혹은 노이즈(noise)라 부름
- 혹은, 어떤 클러스터는 다른 클러스터에 일부로 볼 수도 있음
= 어떤 클러스터 내에서 클러스터링 다시 실행 가능
→ “클러스터가 계층을 가졌다”라고 말함

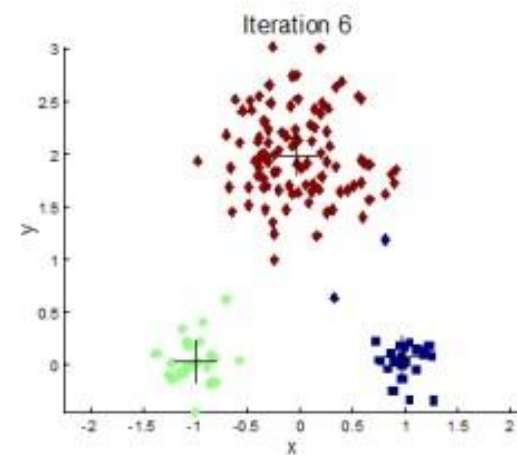
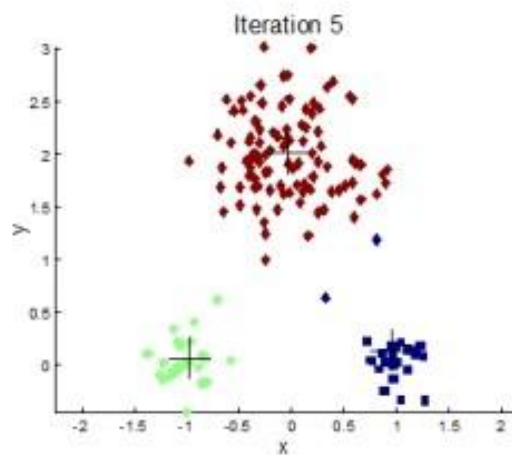
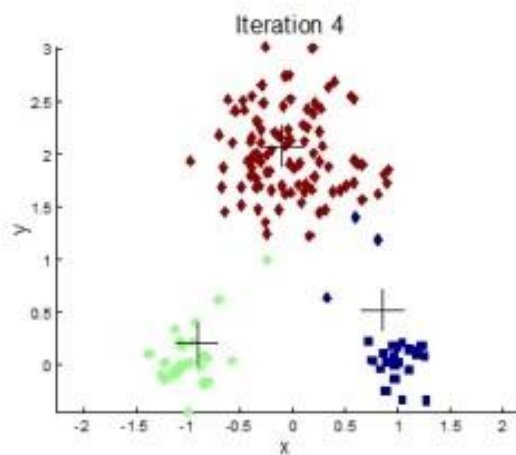
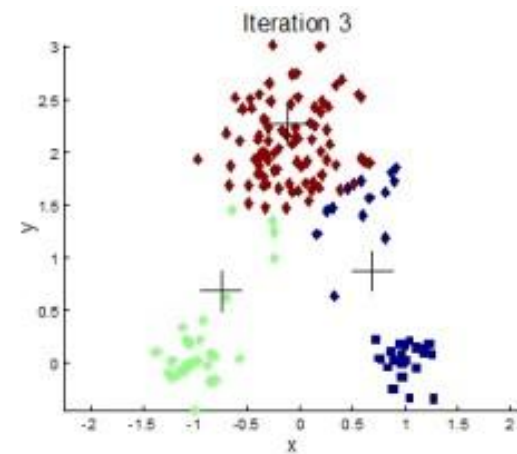
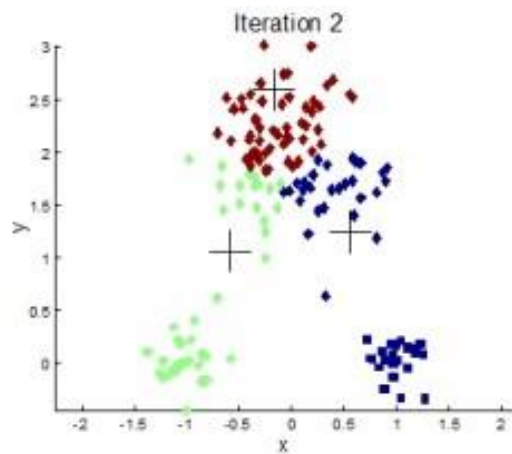
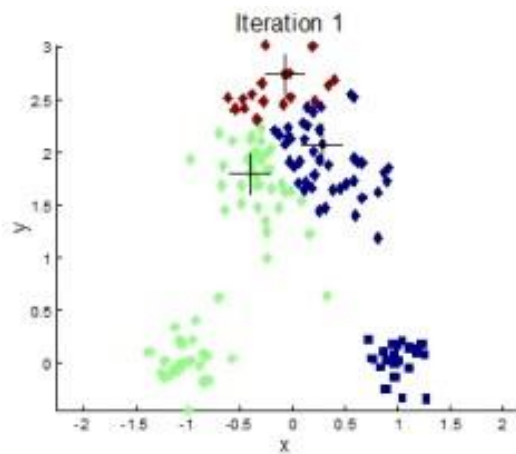
K-Means 클러스터링

- 대표 클러스터링 알고리즘 중 하나
- k 개의 클러스터에 데이터를 할당하는데, 다음 1, 2 단계를 반복적으로 수행하며 이뤄짐
 - 0단계: 임의로 k 개의 데이터 포인트를 선택하여 클러스터로 간주
 - 1단계: 각 데이터 포인트와 클러스터 중심 사이의 거리 계산
 - 클러스터 중심은 클러스터 내 데이터 포인트 평균 (Mean)
 - 2단계: 업데이트 된 데이터 포인트들로 클러스터 중심 다시 계산
 - 종료 : $WSSSE$ (혹은 $WCSS$) 값에 큰 변동이 없으면 종료

클러스터 성능 평가

- WSSSE (Within Set Sum of Squared Error)
 - 클러스터 내부에 있는 데이터 간의 거리를 모두 합한 값
- WCSS (Within Cluster Sum of Squares)
 - 각 데이터 포인트와 클러스터 중심 간의 거리에 대한 제곱 오차 합 (Sum of Squares)

K-Means 클러스터링



K-Means 클러스터링 in Python

- 사이킷런(scikit-learn)의 KMeans 클래스 사용

<KMeans 클래스 생성자 파라미터>

Constructor parameter	Default	Example values	Description
<code>n_clusters</code>	8	3, 36	The number of clusters to determine
<code>max_iter</code>	300	200, 37	Maximum number of iterations for a single run
<code>n_init</code>	10	8, 10	Number of times to rerun the algorithm with different seeds
<code>tol</code>	1e-4	1e-3, 1e-2	Value regulating stopping conditions

K-Means 클러스터링 in Python

- Newsgroup의 포스트들을 클러스터링 (전처리 적용)
 - 필요한 Python Module들 Import!

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.datasets import fetch_20newsgroups
from nltk.corpus import names
from nltk.stem import WordNetLemmatizer
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
```

K-Means 클러스터링 in Python

- Newsgroup의 포스트들을 클러스터링 (전처리 적용)
 - 데이터 전처리 과정 (앞과 동일)

```
cv = CountVectorizer(stop_words="english", max_features=500)
groups = fetch_20newsgroups()
cleaned = []
all_names = set(names.words())
lemmatizer = WordNetLemmatizer()

for post in groups.data:
    lemmatized_list = [ lemmatizer.lemmatize( word.lower() )
                        for word in post.split()
                        if word.isalpha() and word not in all_names ]
    cleaned.append(' '.join(lemmatized_list))

transformed = cv.fit_transform(cleaned)
```

K-Means 클러스터링 in Python

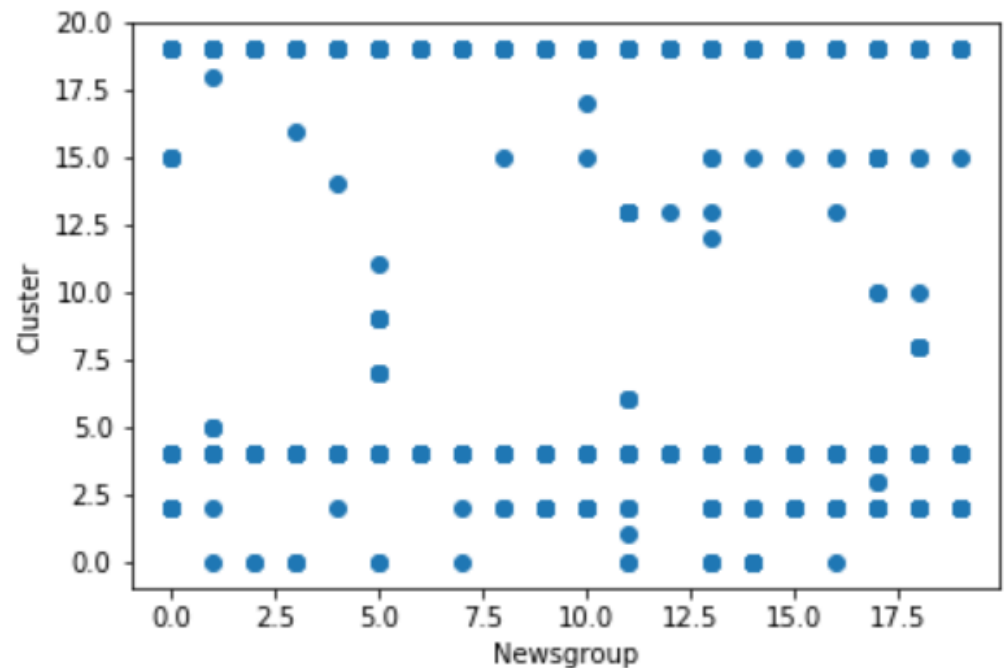
- Newsgroup의 포스트들을 클러스터링 (전처리 적용)
 - K means 클러스터링 수행

```
km = KMeans(n_clusters=20)  
km.fit(transformed)
```

K-Means 클러스터링 in Python

- Newsgroup의 포스트들을 클러스터링 (전처리 적용)
 - 결과 시각화

```
labels = groups.target  
plt.scatter(labels, km.labels_)  
plt.xlabel('Newsgroup')  
plt.ylabel('Cluster')  
plt.show()
```



Lab 5-1: K-Means Algorithm

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Dimensionality Reduction

차원의 저주 Curse of Dimensionality

- 앞서 피쳐 엔지니어링을 통해 여러 피쳐들을 새로 생성하여 기존 학습 모델을 개선하는 것을 확인하였다.
- 하지만 피쳐가 항상 많다고 높은 성능이 보장되는 것은 아니다.
- 이렇게 차원을 늘려도(= 피쳐 수를 늘려도) 성능이 높아지지 않는 현상(심지어는 더 떨어질 수도 ...)을 “차원의 저주”라고 부른다.

주된 이유는 피쳐 수가 늘어나면 확장된 피쳐 공간을 커버할 수 있을 정도의 데이터가 그만큼 더 많이 있어야 하기 때문이다.

차원 감소 Dimensionality Reduction

- 이번 강의에서는 피처 엔지니어링과는 정반대 접근 방법인 차원 감소를 알아본다.
- **차원 감소**는 데이터의 가치 있는 부분은 유지하면서 불필요한 부분을 학습 전에 미리 제거하는 것을 말한다.
- 차원 감소 방법의 두 가지 분류
 - 피처 선택 Feature Selection
 - 피처 추출 Feature Extraction

차원 감소 Dimensionality Reduction

■ 피처 선택 Feature Selection

- 기존 피처들 중에 상대적으로 더 가치있는 일부 피처를 선택하는 것
- 예) 상관 관계, 상호 정보

■ 피처 추출 Feature Extraction

- 기존 피처 공간을 더 낮은 차원의 피처 공간으로 변환
- 예) 주성분 분석 (PCA, Principle Component Analysis)
선형 판별 분석 (LDA, Linear Discriminant Analysis)
다차원 축적 (MDS, Multidimensional Scaling)

Lab 5-2: Dimensionality Reduction

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Feature Selection - Correlation

상관 관계 Correlation

- 상관 관계를 통해 피처 간의 의존성을 확인할 수 있다.
- 대표적인 상관 관계 척도로 Pearson Correlation이 있다.
- Pearson Correlation은 두 피처(변수) 간 선형 의존성(선형 관계)의 존재 여부 및 강도를 알려준다.
 - -1 ~ 1 사이 값을 가지며, 0에 가까울 수록 상관 관계가 없음을 의미하고
-1로 갈 수록 음의 상관관계(한 쪽이 올라가면 다른 쪽이 감소)
1로 갈 수록 양의 상관관계(한 쪽이 올라가면 다른 쪽도 증가)를 나타낸다.

$$Cor(X_1, X_2) = \frac{Cov(X_1, Y_1)}{\sigma_{X_1} \sigma_{X_2}}$$

$$Cov(X_1, X_2) = E[(X - E[X])(Y - E[Y])]$$

공분산

Pearson Correlation in Python

- `scipy.stats`의 `pearsonr` 함수를 사용한다.
- 상관도를 알아보고 싶은 한 쌍의 피처를 입력하면 **Correlation Coefficient (상관계수)**와 **p-value**가 튜플로 반환
 - (복습) P-value는 두 피처의 상관 관계는 없다는 Null Hypothesis를 참이라 가정했을 때 지금과 같은 입력 피처 값들을 얻을 수 있을 확률을 의미
→ 보통 p값이 0.05 보다 작으면 Null Hypothesis 를 Reject 한다.

```
from scipy.stats import pearsonr
```

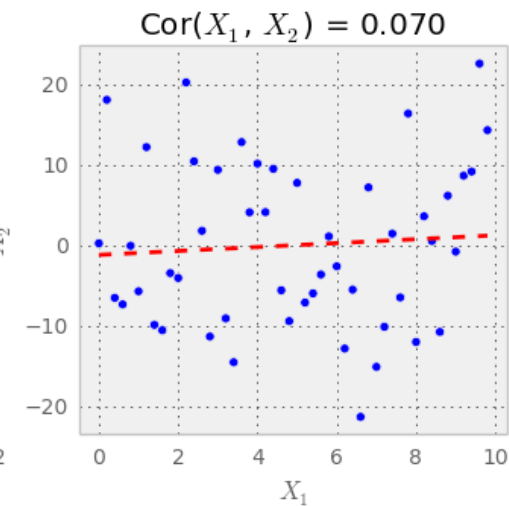
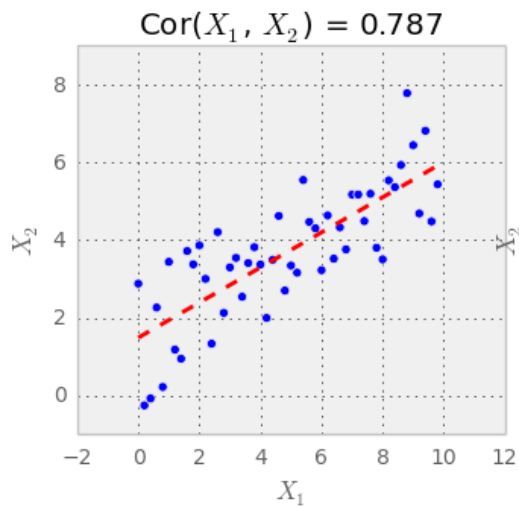
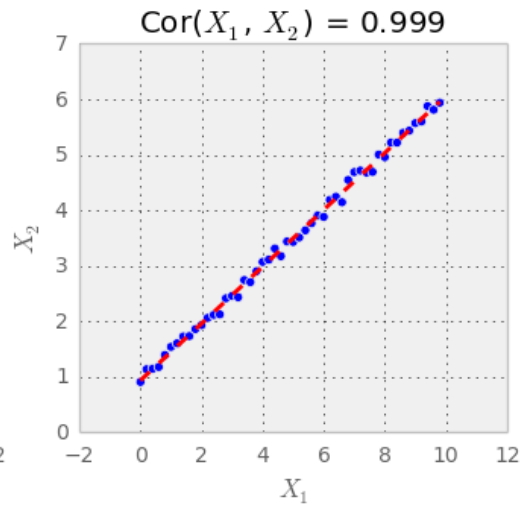
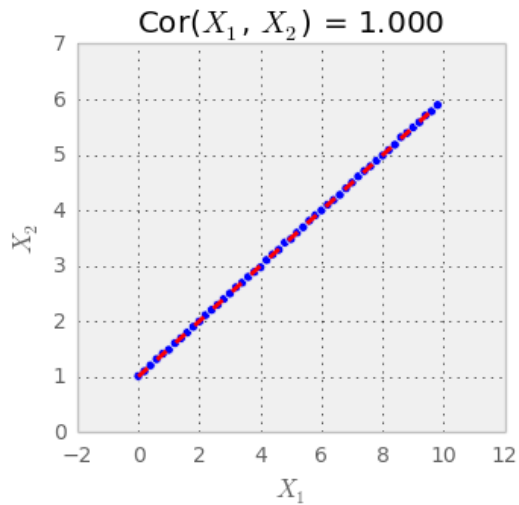
```
pearsonr([1, 2, 3], [1, 2, 3.1])
```

```
(0.9996222851612184, 0.017498096813278487)
```

```
pearsonr([1, 2, 3], [1, 20, 6])
```

```
(0.25383654128340477, 0.8366149366822743)
```

상관 관계 예



처음 3개 그래프는

상관계수가 높은 반면

마지막 그래프는

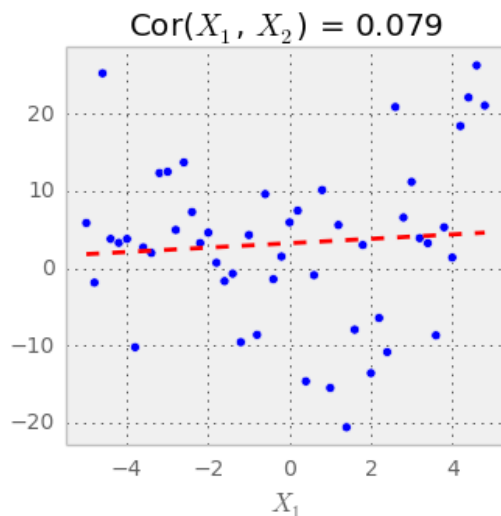
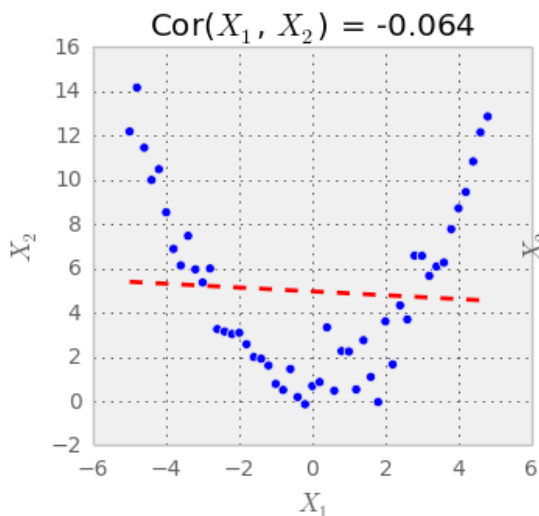
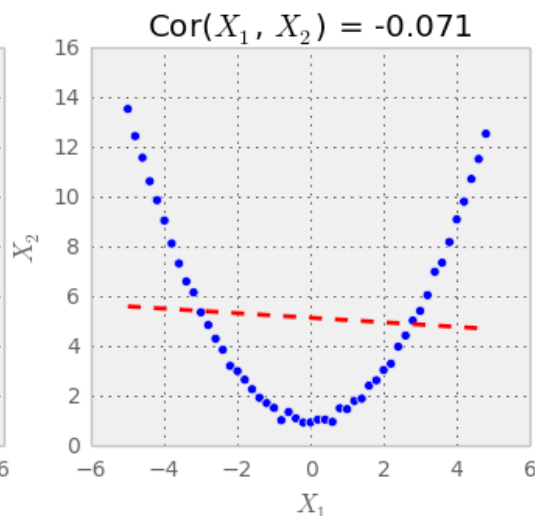
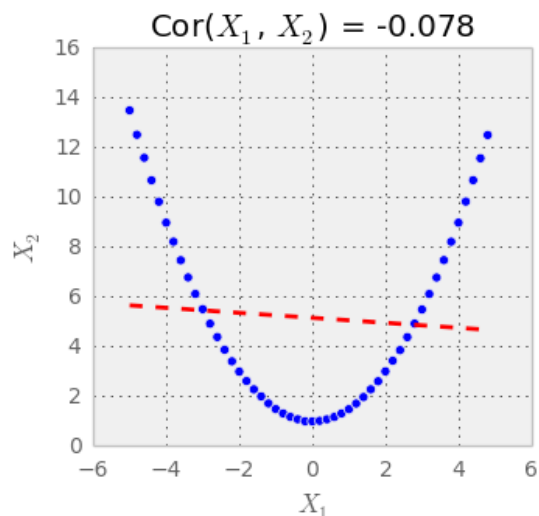
상관계수가 작은 편

Pearson Correlation

- 상관 관계를 토대로 중복된 피처들을 골라내며 피처를 선택할 수 있다.
- 상관 관계에 따른 피처 선택
 - 일단, p-value가 0.05 미만인 의미 있는 상관관계에 주목
 - 그 중에서도 Correlation Coefficient가 높은 관계에 주목
 - 상관 관계가 높은 두 변수(피처) 중 하나를 선택

(회귀 분석의 경우에는 Y와 상관 관계가 낮은 피처를 제거하는 방식으로 활용 가능하다.)

비선형 상관관계 Non-linear Correlation



Pearson Correlation은
선형 관계를 측정하는 지표다.

➔ 만약 두 피처가 비선형 관계라면
Pearson Correlation의
Coefficient 값도 작게 나올 것

➔ 비선형 상관 관계를 측정하는
다른 지표를 사용해야 한다.
(예: Spearman's Rank Correlation)

Feature Selection – Mutual Information

상호 정보 Mutual Information

- 상관 관계를 사용한 방법은
데이터가 선형일지 비선형일지 미리 판단하는 것이 필요
- 하지만 다르게 생각해보면
피처를 선택하는데 있어서 그들이 서로 어떤 관계일지까지
면밀히 알 필요는 없다.
- 각 피처가 **모델 예측 성능에 얼마나 많은 기여**를 하는지가
결과적으로 중요
(즉, 예측을 잘 할 수 있도록 도와주는 정보를 다른 피처와 겹치지 않고
얼마나 고유하게 제공하고 있는가가 중요)

상호 정보 Mutual Information

- 상호 정보는 엔트로피에 기반한다.
 - 정보가 많으면 → 복잡도는 떨어진다.
 - 정보가 적으면 → 복잡도는 높아진다. (Random)

$$Entropy = - \sum_{k=1}^K f_k * \log_2 f_k \quad (f_k \text{는 } k\text{번째 클래스에 해당하는 데이터의 비율})$$

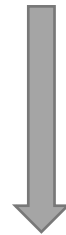
두 개 클래스가 존재할 때,

- 모든 데이터가 한 클래스에만 속한다면 엔트로피는 $-(1 * \log_2 1 + 0) = 0$
- 두 클래스에 반반씩 속한다면 엔트로피는 $-(0.5 * \log_2 0.5 + 0.5 * \log_2 0.5) = 1$

상호 정보 Mutual Information

- 엔트로피가 하나의 변수(피처)의 정보량을 측정한다면
상호 정보는 두 변수(피처들)의 정보량을 측정한다.

$$I(X, Y) = \sum_{i=1}^m \sum_{j=1}^n P(X_i, Y_j) * \log_2 \frac{P(X_i, Y_j)}{P(X_i)P(Y_j)}$$



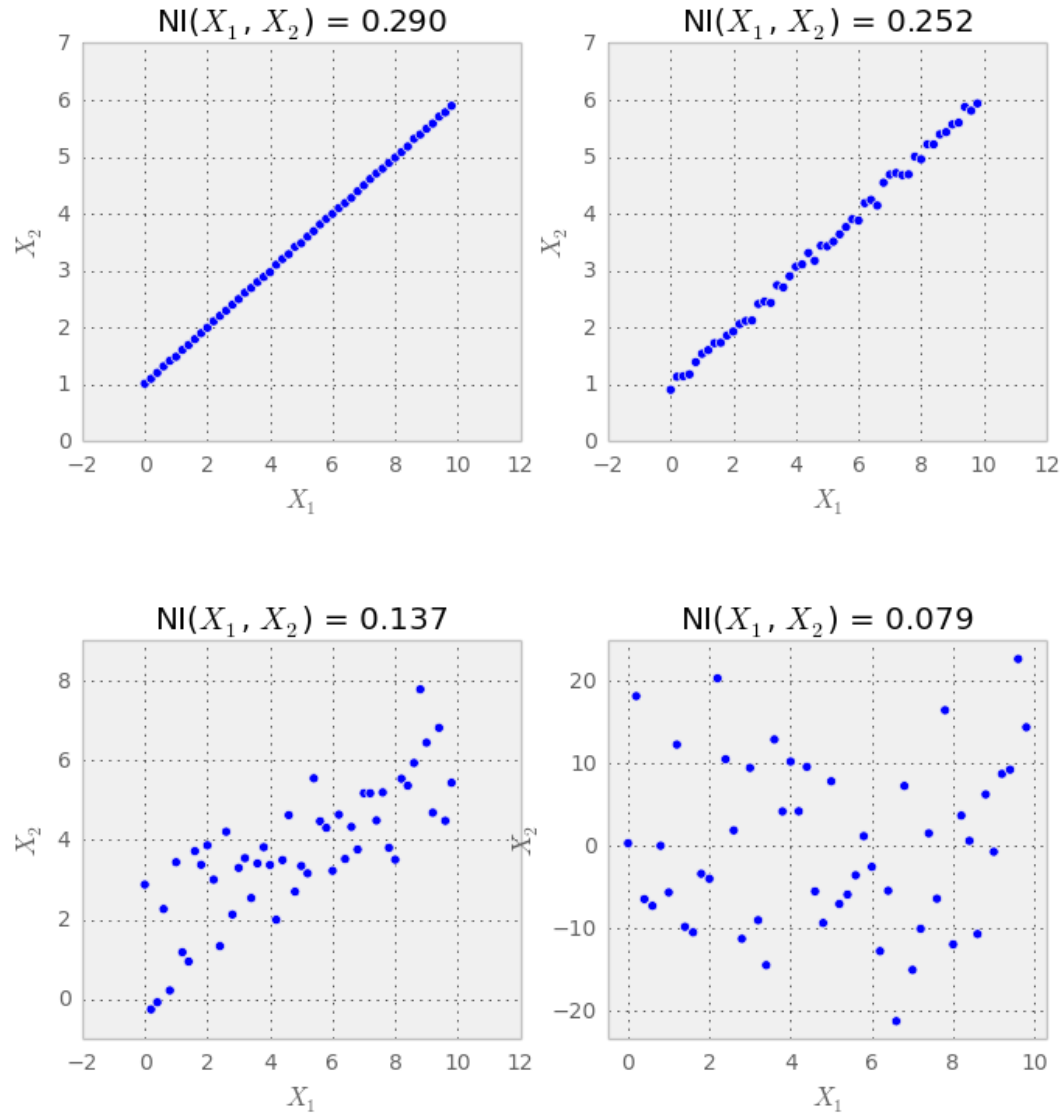
0~1 사이에 값을 만들기 위해
정규화를 사용하기도 한다.

$$NI(X, Y) = \frac{I(X, Y)}{H(X) + H(Y)}$$

값이 작으면 두 변수가 제공하는 정보량은 많다. vs. 값이 크면 두 변수가 제공하는 정보량은 작다.

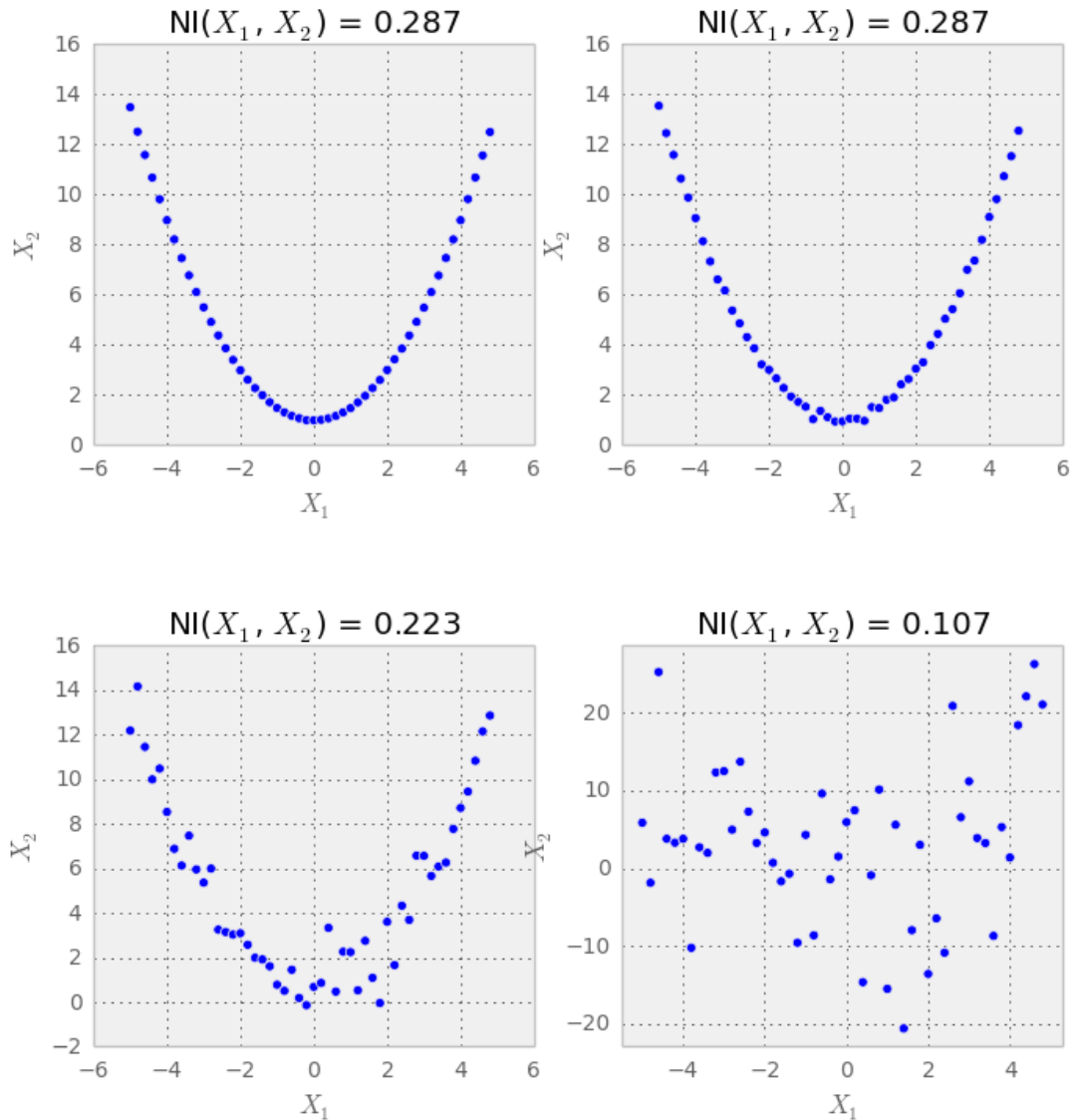
상호 정보 Mutual Information

<선형 관계>



상호 정보 Mutual Information

<비선형 관계>



상호 정보 Mutual Information

- 상관 관계와 같은 방식으로
상호 정보를 토대로 중복된 피처들을 골라내며 피처를 선택할 수 있다.
- 상호 정보에 따른 피처 선택
 - 상호 정보 값이 높은 쌍에 주목
 - 상호 정보 값이 높은 두 변수(피처) 중 하나를 선택

(회귀 분석의 경우에는 Y와의 상호정보 값이 낮은 피처를 제거하는 방식으로 활용 가능하다.)

Feature Selection - RFE

피처 선택 Feature Selection

- 상관 관계나 상호 정보를 사용하는 방법 모두 피처들을 한 쌍씩 선택해 값을 계산해야 하기 때문에 몇가지 문제가 발생할 수 있다.
 - 두 개씩 봐야 하는 조합이 많을 수 있다. → 많은 연산
 - 개별적으로 선택했는데, 전체를 다 모아놓고 보면 모델의 예측 성능에는 좋지 않을 수 있다.

이를 보완하기 위해 모델 학습 과정에서
정확도와 같은 성능 지표를 함께 보면서 피처를 선택해 가는 방법도 있다.

대표적 알고리즘 : Recursive Feature Elimination

RFE Recursive Feature Elimination

- RFE는 전체 피처들을 모두 사용하여 일단 학습 모델을 만들고 학습된 모델에 대하여 각 피처가 기여하는 정도를 평가하여 (피처를 제외했을 때 성능 변화로 평가) 기여도가 높은 피처들을 선택하고 선택된 피처들로 다시 모델을 학습하는 방법

RFE Recursive Feature Elimination

- RFE 테스트를 위한 데이터 생성 코드

```
from sklearn.datasets import make_classification  
  
X,y = make_classification(n_samples=100, n_features=10,  
                          n_informative=3, random_state=0)
```

- n_samples : 만들어 낼 샘플 수
- n_features : 만들어 낼 피쳐들 수
- n_informative : 의미있는 (제공 정보량이 많은) 피쳐 수
- n_classes : 만들어 낼 클래스 수 (default = 2)

RFE Recursive Feature Elimination

- 전체 피처를 다 사용하여 Logistic Regression 모델을 만들고
RFE를 사용해 3개 피처를 선택한 뒤
선택된 피처들로 다시 모델을 학습시키는 코드

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X, y)

selector = RFE(clf, n_features_to_select=3)
selector = selector.fit(X, y)
```


RFE Recursive Feature Elimination

- 피쳐들 중 어떤 것이 선택됐는지(support_)
- 피쳐들 중 기여도 순위가 어떠했는지(ranking_)를 볼 수 있다.

```
print(selector.support_)
```

```
[False False  True False False False  True  True False False]
```

```
print(selector.ranking_)
```

```
[5 4 1 2 6 7 1 1 8 3]
```

RFE Recursive Feature Elimination

- `n_features_to_select` 값을 바꾸면서 어떤 피처들이 선택되는지 보면 안정적으로 피처가 잘 선택되는 것을 볼 수 있다.

<code>n_features_to_select</code>	<code>support_</code>	<code>ranking_</code>
1	[False False False True False False False False False False]	[6 3 5 1 10 7 9 8 2 4]
2	[False False False True False False False False True False]	[5 2 4 1 9 6 8 7 1 3]
3	[False True False True False False False False True False]	[4 1 3 1 8 5 7 6 1 2]
4	[False True False True False False False False True True]	[3 1 2 1 7 4 6 5 1 1]
5	[False True True True False False False False True True]	[2 1 1 1 6 3 5 4 1 1]
6	[True True True True False False False False True True]	[1 1 1 1 5 2 4 3 1 1]
7	[True True True True False True False False True True]	[1 1 1 1 4 1 3 2 1 1]
8	[True True True True False True False True True True]	[1 1 1 1 3 1 2 1 1 1]
9	[True True True True False True True True True True]	[1 1 1 1 2 1 1 1 1 1]
10	[True True True True True True True True True True]	[1 1 1 1 1 1 1 1 1 1]

(참고) L1 정규화를 통한 피처 선택

- 방금 봤던 피처 선택 방법들은
학습 전 (상관관계, 상호 정보) 혹은 학습 후 (RFE)에
피처를 선택한다.
- 이전에 배웠던 L1 정규화는 **학습 중**에 피처를 선택한다.

(참고) 정규화 Regularization

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \alpha \|\mathbf{w}\|^q$$

- α 는 정규화를 얼마나 목적함수에 반영할지 조절하는 파라미터
- 정규화는 벡터의 Norm(크기)를 보통 사용

- L1 정규화 : L1 Norm을 사용

$$\|\mathbf{w}\|^1 = \sum_{i=1}^k |w_i|$$

- L2 정규화 : L2 Norm을 사용

$$\|\mathbf{w}\|^2 = \sum_{i=1}^k w_i^2$$

Feature Extraction

차원 감소 Dimensionality Reduction

■ 피처 선택 Feature Selection

- 기존 피처들 중에 상대적으로 더 가치있는 일부 피처를 선택하는 것
- 예) 상관 관계, 상호 정보

■ 피처 추출 Feature Extraction

- 기존 피처 공간을 더 낮은 차원의 피처 공간으로 변환
- 예) 주성분 분석 (PCA, Principle Component Analysis)
선형 판별 분석 (LDA, Linear Discriminant Analysis)
다차원 축적 (MDS, Multidimensional Scaling)

피처 추출 Feature Extraction

- 상황 1 : 불필요한 피처들을 제거했음에도 불구하고 여전히 피처들의 수가 많을 수도 있다.
- 상황 2 : 피처 공간을 가시화 하고 싶은데 차원이 너무 커서 표현이 불가능하다.
(가시화를 위해서는 3차원 이하의 피처 공간이 필요)

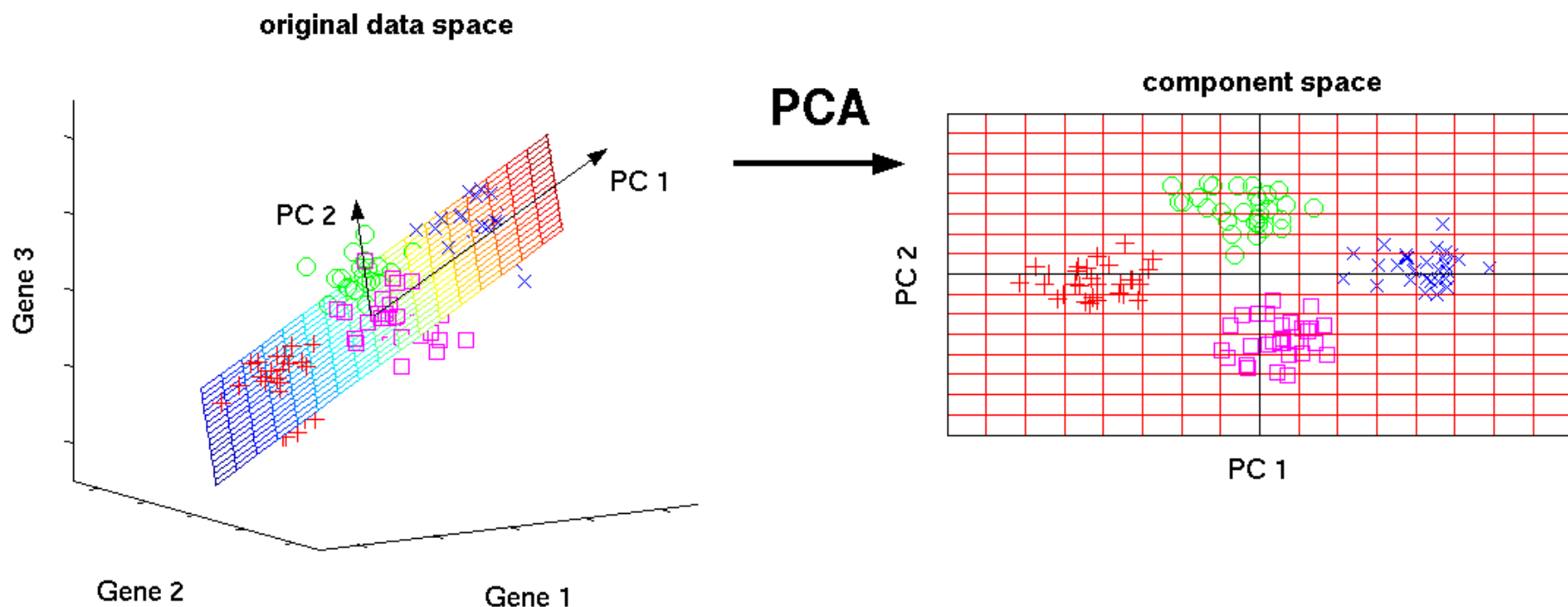
이렇게 기존의 차원으로는 한계가 있을 때
사용할 수 있는 방법으로 피처 추출이 있다.

주성분 분석 Principle Component Analysis

- 대표적인 피쳐 추출 방법
- 특이값 분해 원리에 기반
(Singular Value Decomposition)
- 비지도학습 방법
(즉, 타겟 레이블 없이 수행됨)

주성분분석 Principle Component Analysis

데이터의 분산을 어느정도 유지하면서 기저(차원)을 줄이는 방법



예) 데이터의 설명력(= 분산)은 대략적으로 유지하면서
3차원에서 2차원으로 차원 변환

주성분 분석 in Python

```
import numpy as np
x1 = np.arange(0, 10, .2)
x2 = x1 + np.random.normal(loc=0, scale=1, size=len(x1))

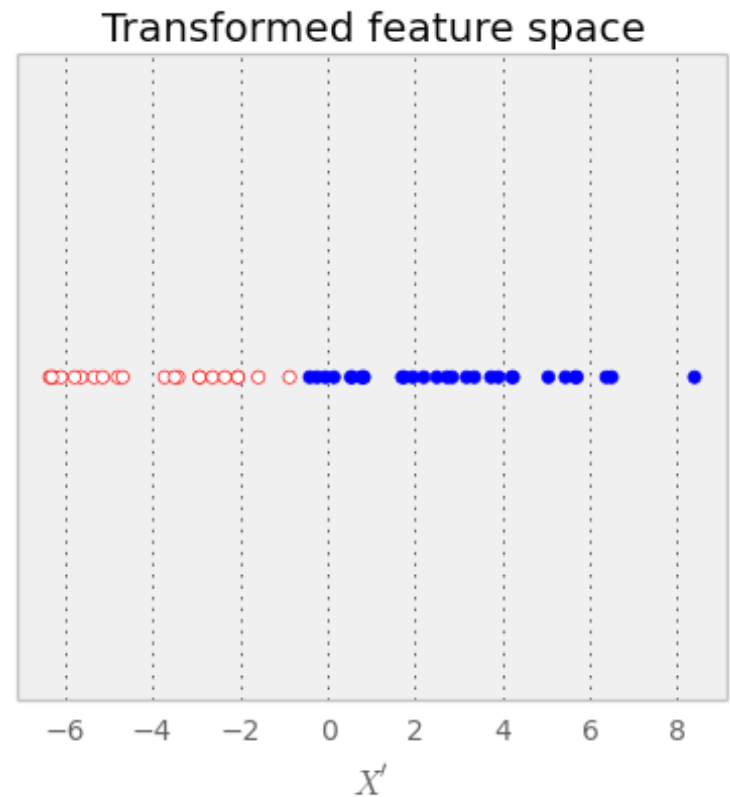
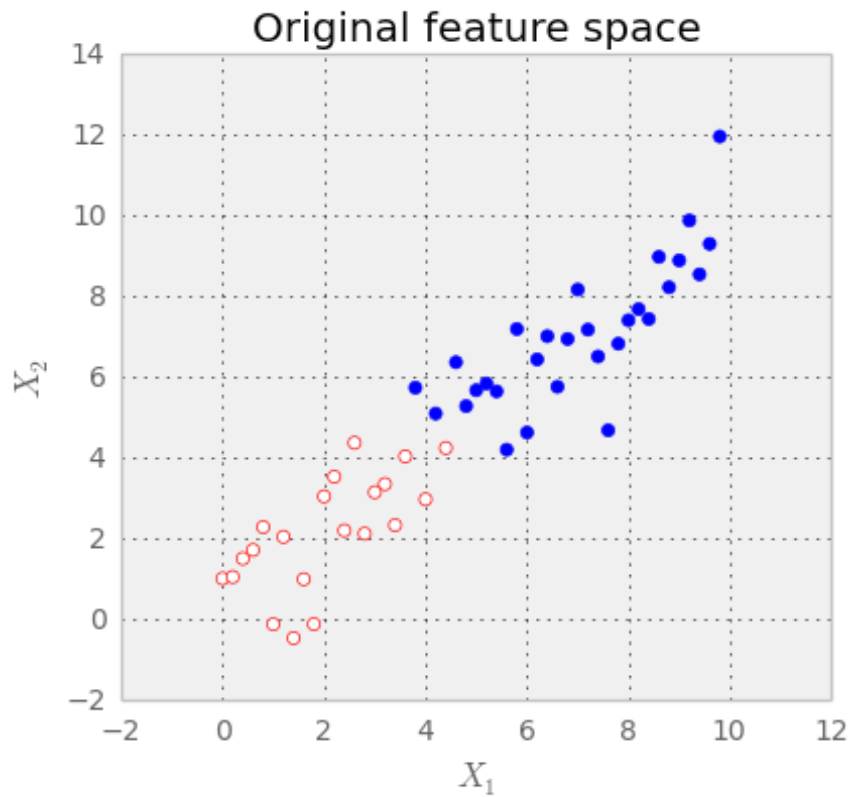
X = np.c_[x1, x2]
good = (x1>5) | (x2>5) # some arbitrary classes
bad = ~good # to make the example look good

from sklearn import linear_model, decomposition, datasets
pca = decomposition.PCA(n_components=1)
Xtrans = pca.fit_transform(X)

print(pca.explained_variance_ratio_)
```

```
[0.97922]
```

주성분 분석 in Python

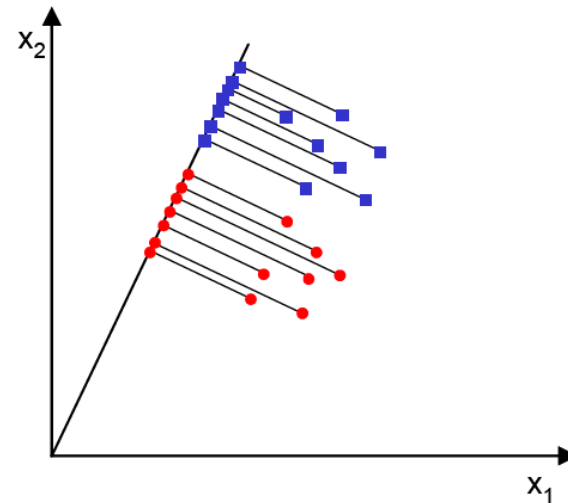
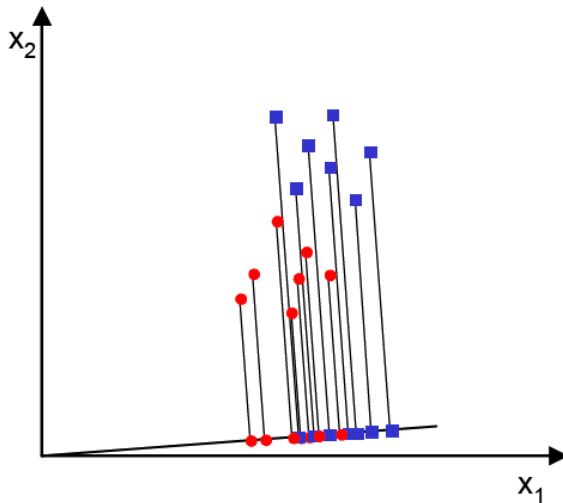


LDA Linear Discriminant Analysis

- 데이터를 더 낮은 차원의 공간에 투영했을 때

(= 더 낮은 차원의 공간에 대한 사영을 구했을 때)

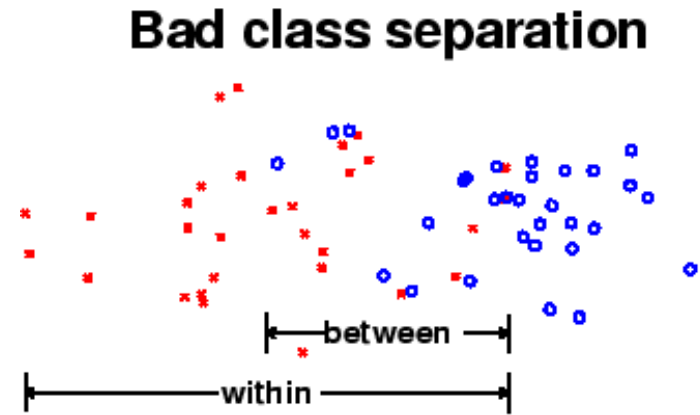
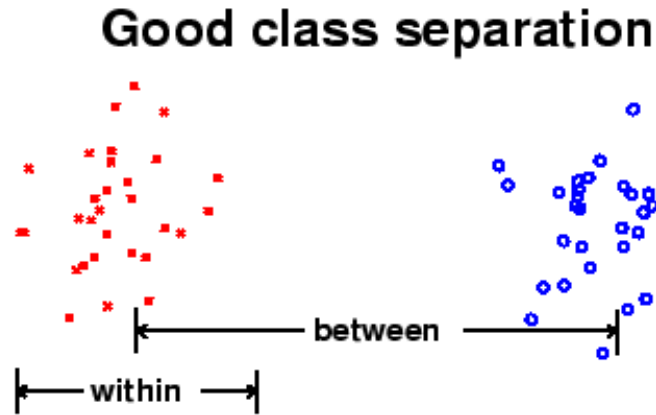
클래스 별로 데이터가 잘 분리되는 공간을 찾는 방법



이차원 공간의 데이터의 사영을 구할 수 있는 일차원 공간은 다양

LDA Linear Discriminant Analysis

- 데이터가 잘 분리 된 공간은 어떤 것?



클래스간 분산 (between-class Scatter)
크다 ← ----- → 작다
클래스내 분산 (within-class Scatter)

LDA Linear Discriminant Analysis

- 어떤 선형 공간에 투영된 사영 $\hat{\mathbf{y}}$ 들이 클래스에 따라 잘 분류되는지 평가하기 위해 분산을 사용한다.

$$S = \frac{\sigma_{between}^2}{\sigma_{within}^2}$$

$\sigma_{between}^2$: 클래스 간 $\mathbf{u} \cdot \mathbf{y}$ 값들의 분산

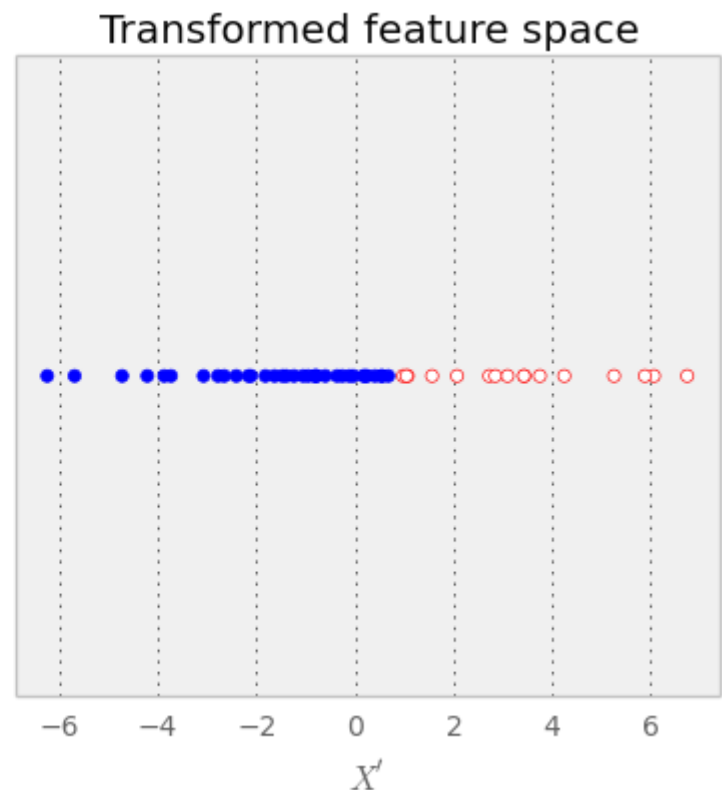
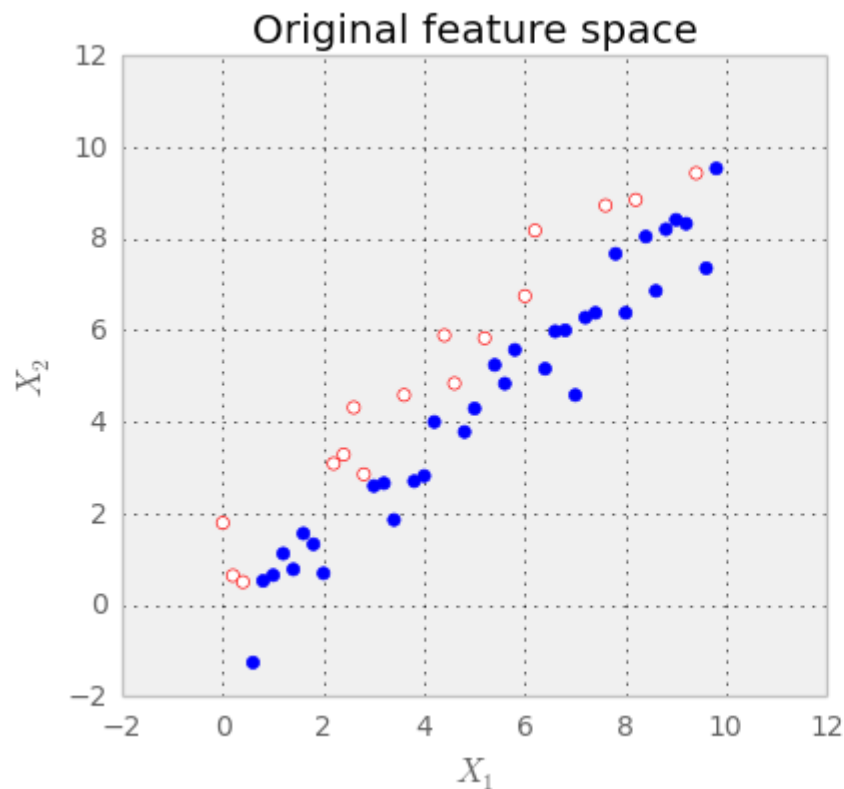
σ_{within}^2 : 클래스 내 $\mathbf{u} \cdot \mathbf{y}$ 값들의 분산

결국 s 가 최대가 되도록 하는 $\mathbf{u}$ 를 찾는 것이 LDA

LDA in Python

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

lda_inst = LDA(n_components=1)
Xtrans = lda_inst.fit_transform(X, good)
```



LDA vs. PCA

- PCA는 비지도학습, LDA 지도학습
 - 샘플이 적을 때에는 보통 PCA가 LDA보다 잘 동작
 - 샘플이 충분한 경우 LDA가 PCA 보다 더 잘 동작
- ➔ 상황에 맞게 선택해서 사용하면 된다.

MDS Multidimensional Scaling

- MDS는 가시화를 위해 많이 사용되는 알고리즘
→ 주로 2, 3차원으로 차원 변환
- 데이터 점 자체를 보는 것이 아니라
데이터들 간의 **거리** 값(유클리디언 거리)을 사용
- 원래 공간에서 **데이터들 간 거리**와
더 낮은 차원의 공간에서 데이터들 간 거리가
최대한 **유사**한 공간으로 차원을 변환

MDS in Python

```
X = np.c_[np.ones(5), 2 * np.ones(5), 10 * np.ones(5)].T  
print(X)
```

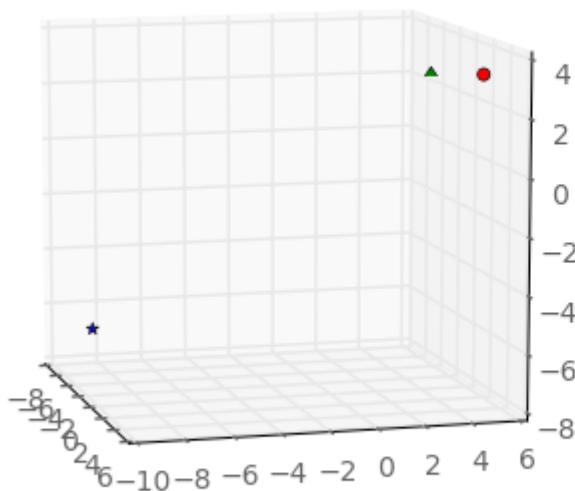
```
[[ 1.  1.  1.  1.  1.]  
 [ 2.  2.  2.  2.  2.]  
 [10. 10. 10. 10. 10.]]
```

```
from sklearn import manifold
```

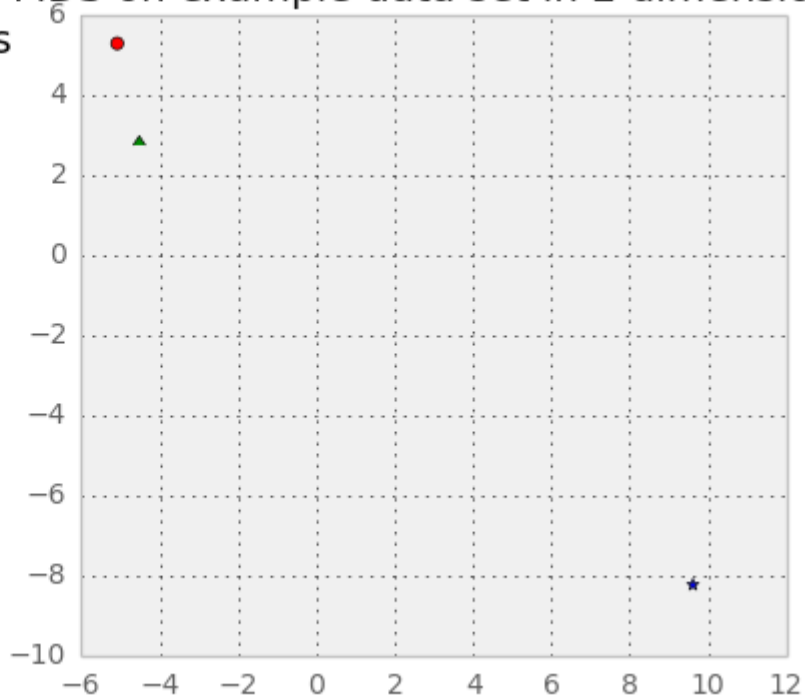
```
mds = manifold.MDS(n_components=3)  
Xtrans = mds.fit_transform(X)
```

MDS in Python

MDS on example data set in 3 dimensions

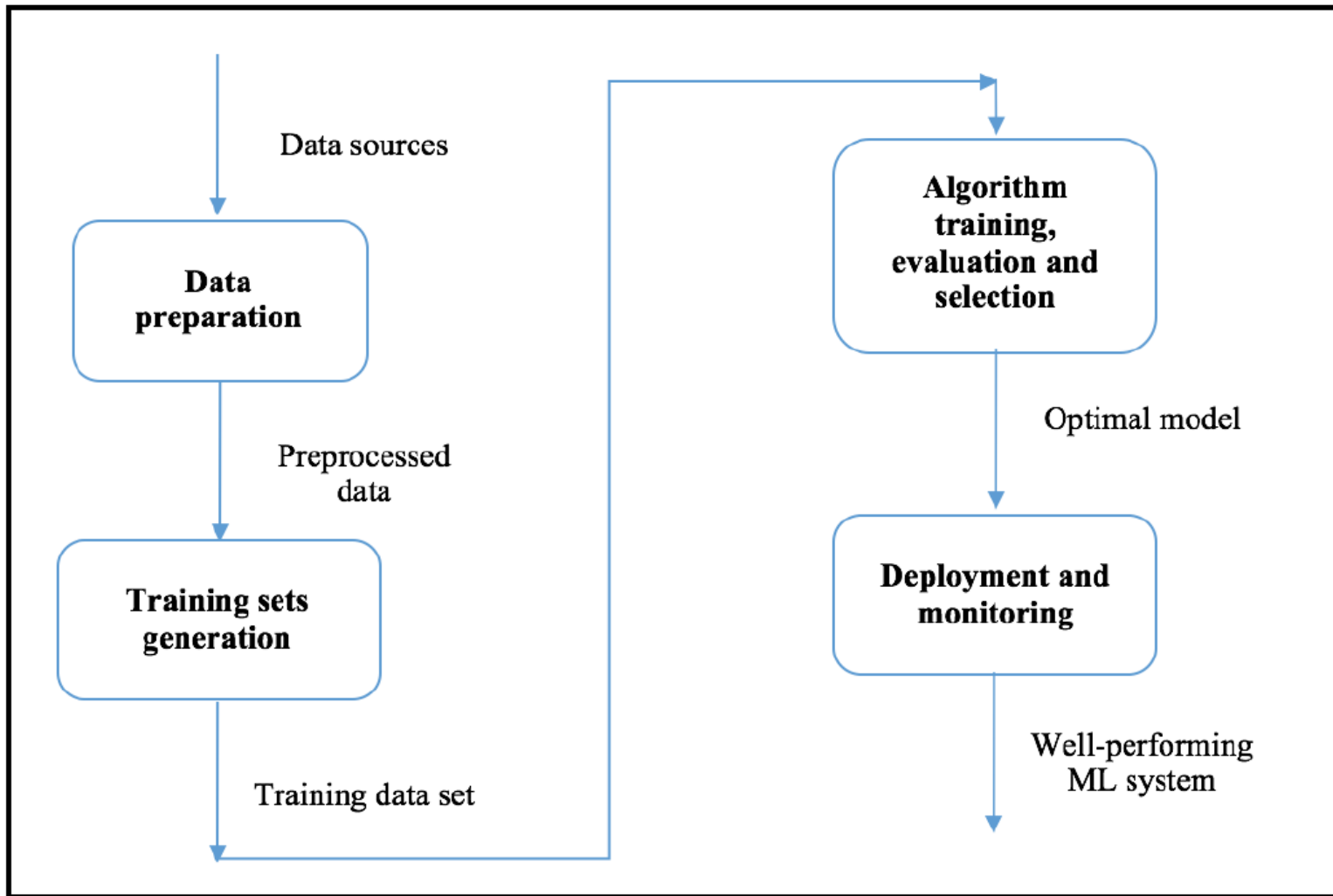


MDS on example data set in 2 dimensions



Best Practices: Data Preparation

ML Workflow



1 – Completely understand the project

- 데이터를 수집하기 전 프로젝트의 목표, 비즈니스 문제가 무엇인지 이해 필요
 - 들여다 볼 데이터의 소스, 충분한 도메인 지식과 전문성이 요구 되는 지 등
- 예: Stock Price Prediction
 - 적합한 데이터는?
 - 무엇을 목표로 하는가?

2 – Collect all fields that are relevant

- 목표를 달성하기 위한 잠재적인 데이터 소스로부터,
 - 가능한 모든 필드 데이터 수집?
 - 중요하다고 여겨지는 서브셋?
- 미리 핵심 지표, 핵심 예측 요소를 알 수 있으면 좋겠지만, 그런 경우는 실제로 거의 없다.
 - 도메인 전문가가 이야기해주는 요소라고 하더라도 과연 가장 좋은 결과를 가져다 줄까?
- 따라서, “가능한 모든” 데이터를 수집하는 것이 좋다.
 - 우리는 어떤 요소가 어떤 역할을 할지 모른다
 - 쓸모 있는 요소 vs. 쓸모 없는 요소?
 - 데이터 수집에 어려움이 있는 요소가 있는 경우가 많이 있어서 고민이 될 수 있음

3 – Maintain consistency of field values

- 사용하고자 하는 데이터를 보면 똑같은 의미인데 다르게 표현된 경우가 많음
 - 특히 여러 데이터를 합쳐서 사용하는 경우
 - 예: Country – USA, US, America -> US
 - 예: Gender – M, Male -> M
- Reformatting도 필요
 - Age 필드에서 21, 35 (ok), 1995, 1994 (x)
 - Rating 필드에서 1,2,3, or one, two, three

4 – Deal with missing data

- 여러가지 이유로 실제 데이터는 대부분 정리가 잘 안되어 있고, 중간에 데이터가 빠져 있거나, 잘못되어 있는 경우가 많음
 - 이런 경우 빈칸, NULL, -1, 999999, unknown 등으로 채워져 있는 경우가 많음
- 결측값의 문제
 - 불완전한 정보 제공
 - 머신러닝 모델에 혼란 야기
 - 모델의 성능에 안좋은 영향을 줄 수 있음

4 – Deal with missing data

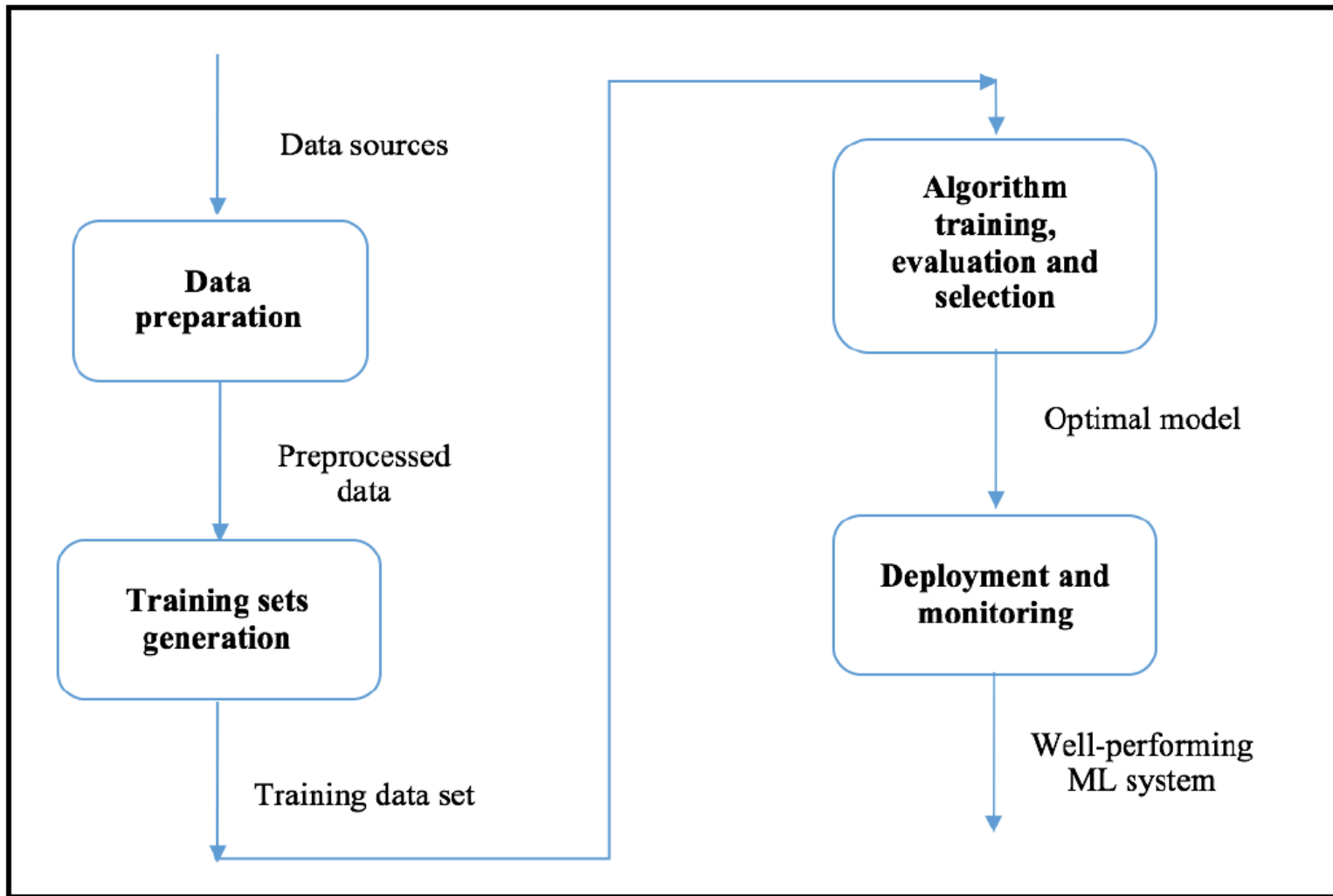
■ 해결방법

- 결측값이 포함된 데이터샘플을 다 버린다
- 데이터 샘플에서 결측값이 포함된 필드만 버린다
- 이미 있는 값을 바탕으로 결측값을 추정한다
 - 결측 데이터 대체(imputation)

■ `from sklearn.preprocessing import Imputer`

Best Practices: Training Sets Generation

ML Workflow



5 – Determine categorical features with numerical values

- 수치형 값을 지닌 범주형 feature 판단
 - 일반적으로 범주형 feature는 위험수준, 직업 및 관심사항과 같은 정성적 정보 전달
 - Feature가 이산형이면서 셀수 있는 숫자 값을 갖게 되면 애매함
- Feature가 수학적 의미를 제공하는지 판단
 - 예: product rating을 1~5까지 매기는 것은 수치형 feature로 볼 수 있음
 - 반대로 월, 요일 등은 범주형 feature로 볼 수 있음

6 – Decide on whether or not to encode categorical features

- Feature가 범주형이면 인코딩을 할지 말지 결정해야 함
- 다음단계에서 어떤 예측 알고리즘을 사용할지에 달려 있음
 - 나이브 베이즈, 트리기반 알고리즘 등 -> 범주형 피쳐 적용 가능
 - 다른 알고리즘 -> 인코딩 필요(예: one-hot encoding)

7 – Decide on whether or not to select features and if so, how

■ Feature selection의 장점

- 불필요한 feature를 제거해서 예측모델의 학습시간 줄여줌
- 오버피팅 줄여줌
- 중요도가 높은 feature 위주의 데이터로 학습이 진행되어서 성능이 높아질 가능성이 있음

■ 항상 feature selection이 성능에 도움이 되지는 않음

- 적용 전, 후의 성능 비교할 필요 있음

8 - Decide on whether or not to reduce dimensionality and if so how

- Dimensionality Reduction의 장점
 - 예측모델의 학습시간 줄여줌
 - 오버피팅 줄여줌
 - 성능이 높아질 가능성이 있음
- 항상 차원 축소가 성능에 도움이 되지는 않음

9 - Decide on whether or not to scale features

- 언제 feature scaling이 필요할까?
- 일반적으로,
 - 나이브 베이즈, 트리기반 알고리즘은 서로 다른 스케일 feature에 영향을 덜 받음
 - Feature들이 서로 독립적이기 때문
 - Logistic or linear regression은 입력 feature에 대한 스케일 영향을 거의 받지 않지만, SGD로 최적화 하는 경우는 주의해야 함
 - SVC, SVR과 같이 샘플간의 거리와 관련된 알고리즘은 스케일되어 있고 표준화된 입력 feature를 필요로 함
 - 최적화를 위해 SGD를 이용하는 경우 꼭 필요함

10 - Perform Feature Engineering With Domain Expertise

- 도메인 지식이 있으면 도메인에 특화된 feature를 만들 수 있음
 - 비즈니스 경험, 비즈니스 인사이트 활용
- 도메인 지식이 성능에 도움이 되는 경우가 많음
 - 반대의 경우도 생길 수 있음
 - 과학적 검증은 항상 중요함

11 - Perform feature engineering without domain expertise

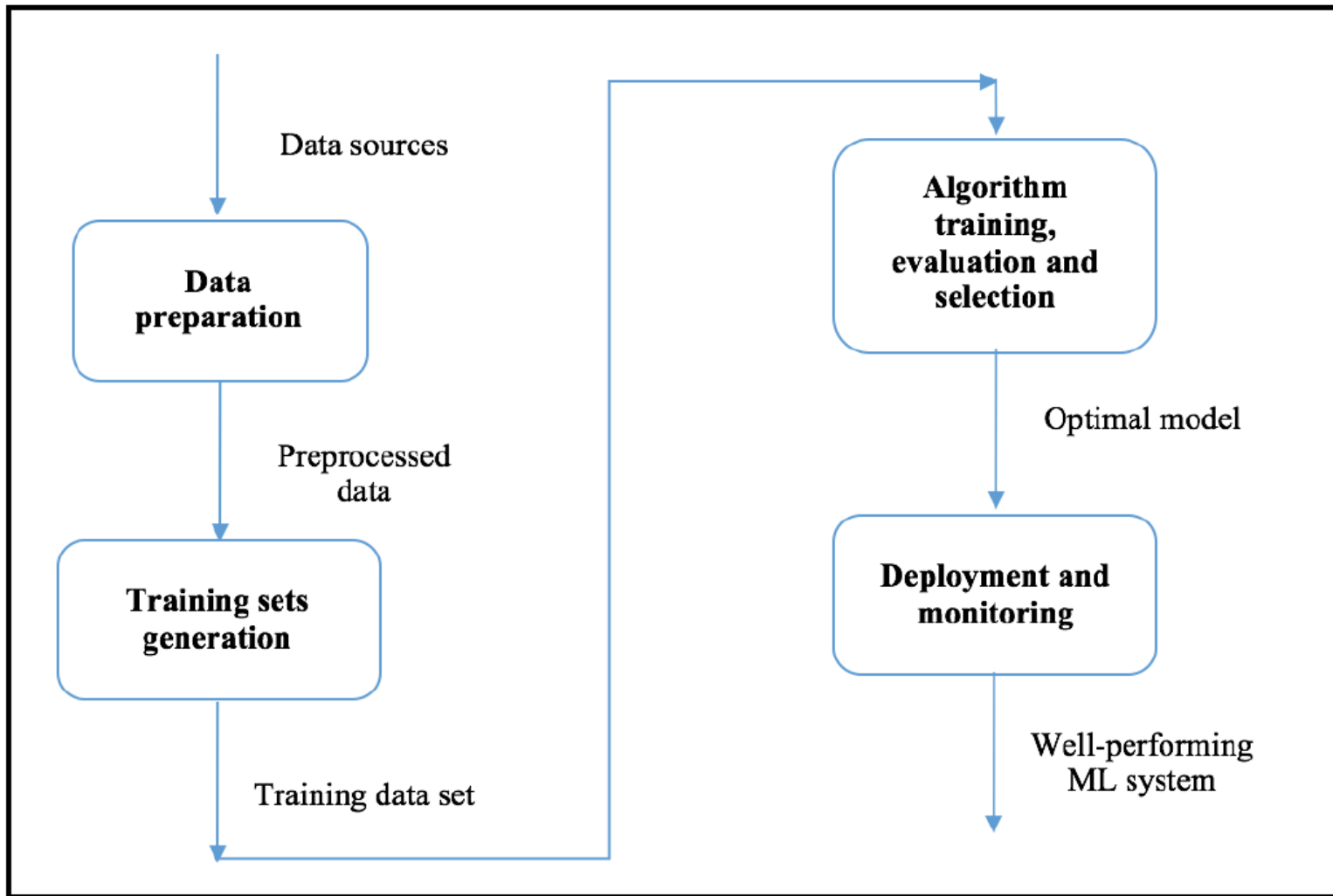
- 어떻게 feature를 만들어 낼 수 있을까?
- 이진화: 수치형 feature를 기준에 따라 둘중 하나의 값으로 변환
 - 예: spam 탐지시 whether "prize" occurs or not
- 이산화: 수치형 feature를 제한된 값을 지닌 범주형 feature로 변환
 - Age -> age group
- 상호작용: 합, 곱 등 두 수치형 feature의 연산, 2개의 범주형 feature에 대한 결합조건 등
- 다항 변환: 다항형 생성 feature
 - 예: a^2 , ab 등

12 – Document how each feature is generated

- Feature를 어떻게 생성했는지 기록 필요
 - 어떻게 확보했는지
 - 어떻게 생성했는지 등
- 대부분 모델 학습단계에서 다시 feature engineering 단계로 돌아오는 경우가 많기 때문에 feature engineering 과정을 명확히 파악할 필요가 있음

Best Practices: Model Training, Evaluation, and Selection

ML Workflow



13 – Choose the right algorithms to start with

- 수많은 알고리즘을 다 테스트하고 파인튜닝하는 것은 많은 시간과 계산 비용이 들어감
- 다음의 사항을 고려할 필요 있음
 - 학습 데이터셋의 크기
 - 데이터셋의 차원
 - 데이터가 선형으로 분리 가능한지 여부
 - feature들이 서로 관련이 없는지 여부
 - 바이어스와 분산간의 trade off
 - 온라인 러닝 필요 여부

13 – Choose the right algorithms to start with

- 나이브 베이즈
- Logistic Regression
- SVM
- Random Forest
- Neural Network

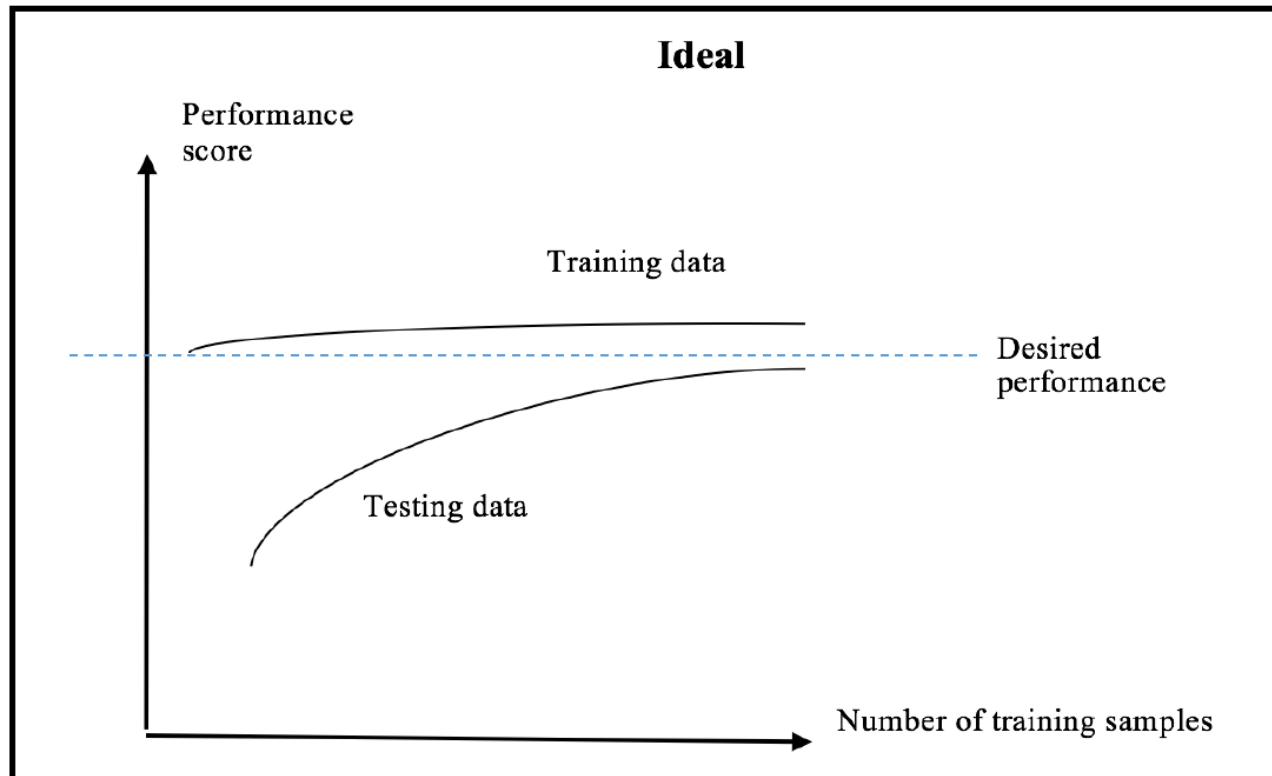
14 – Reduce overfitting

- Cross-validation
- Regularization
- Simplification
- Ensemble learning

15 – Diagnose overfitting and underfitting

■ Learning curve

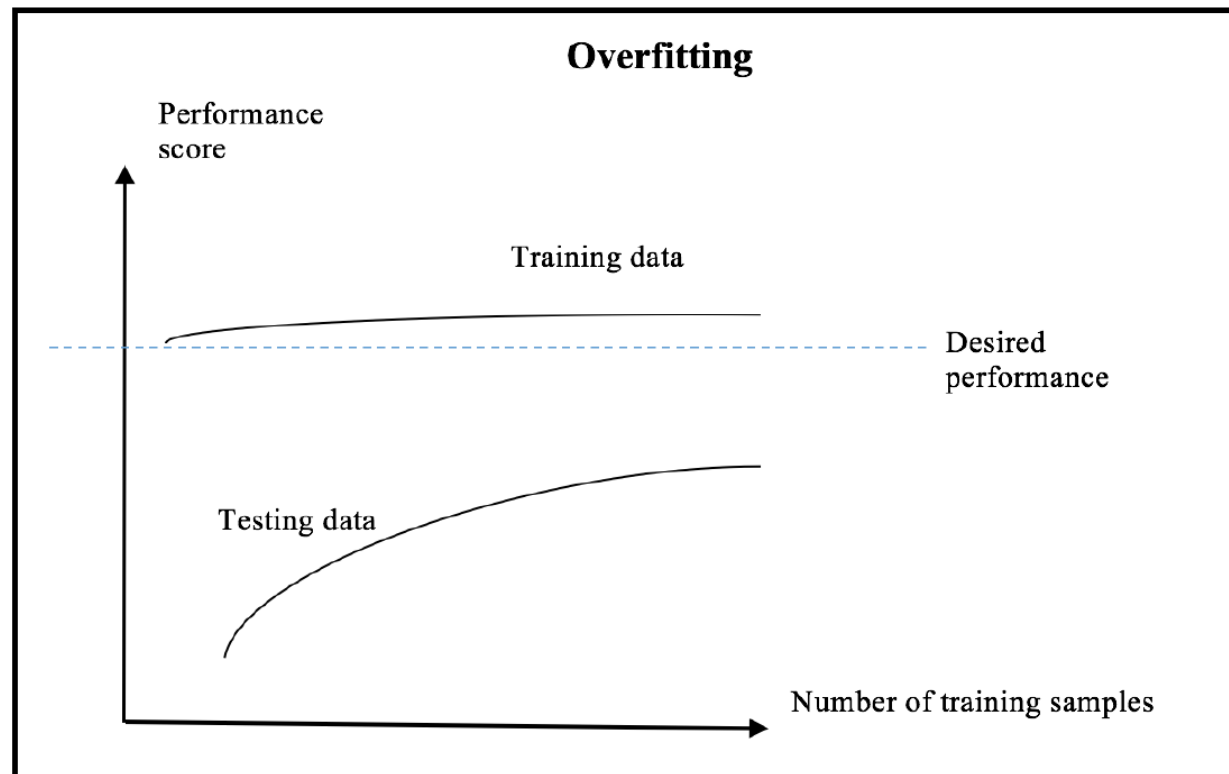
- Compares the cross-validated training & testing scores



15 – Diagnose overfitting and underfitting

■ Learning curve

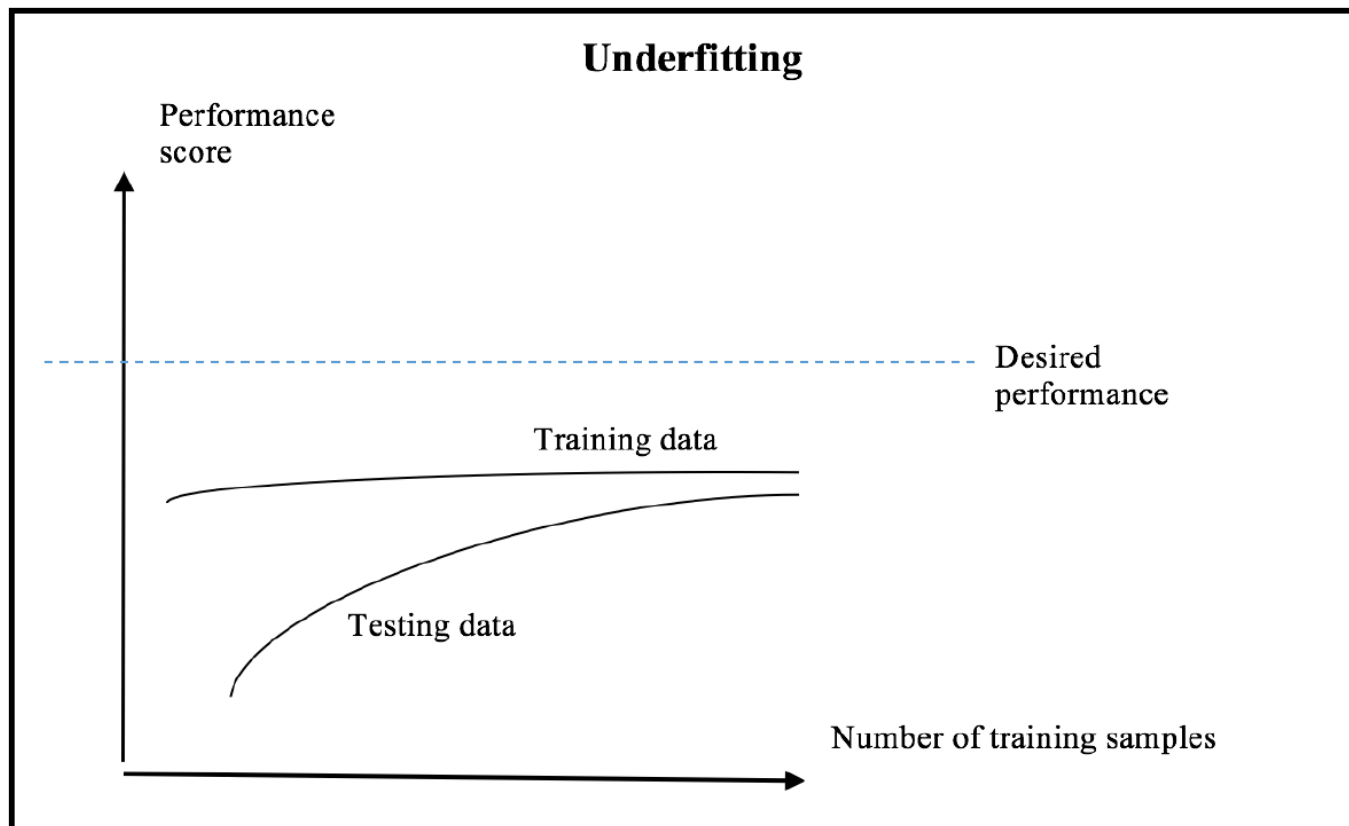
- Compares the cross-validated training & testing scores



15 – Diagnose overfitting and underfitting

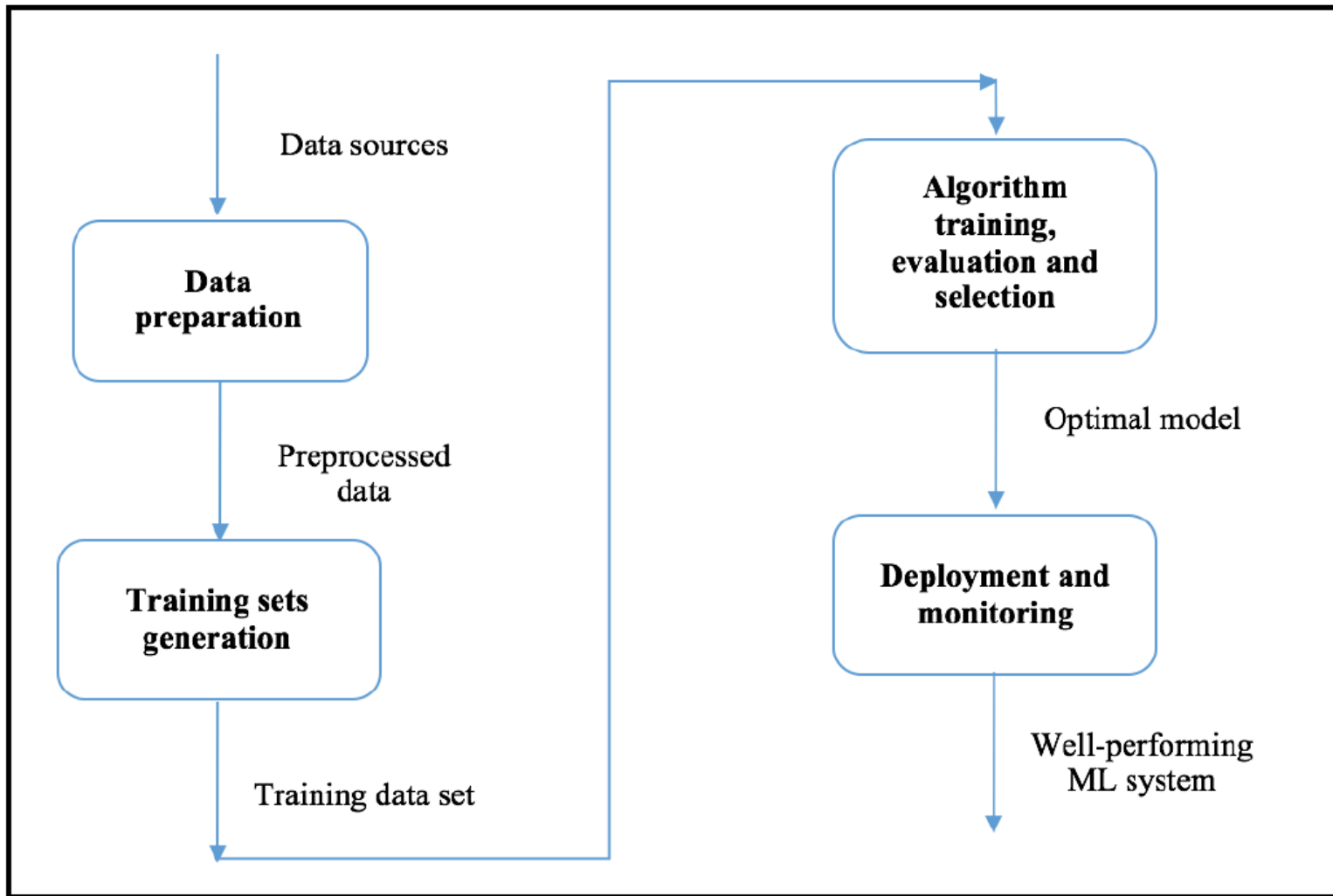
■ Learning curve

- Compares the cross-validated training & testing scores



Best Practices: Deployment and Monitoring

ML Workflow



16 – Save, load, and reuse models

- 만들어진 (학습된) 모델을 저장한 후 배포시에 해당 모델을 불러와서 예측에 활용
- 저장방법: pickle 사용

17 – Monitor model performance

- Log the performance
- Set an alert for a decayed performance
 - 시스템 모니터링 이슈

18 – Update the model

- 성능이 점점 안 좋아지면 데이터의 패턴을 바꿔야 할 때라는 의미
- 모델 업데이트 수행
 - 온라인 러닝이 모델에 맞을 경우 새로운 데이터로 업데이트 할 수 있음
 - 또는 최신 데이터를 이용하여 모델을 완전히 재학습 할 수 있음

Lab 5-3: Competition

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.